

Concurrency, Locks, and Safe Shutdown

Concurrency is about possible interleavings and invariants, not merely adding threads. Start with the state that must remain consistent, choose a synchronization boundary, and define shutdown before optimizing throughput. A correct steady-state loop can still fail during cancellation or partial initialization.

For: Developers preparing for multithreaded systems or backend roles who need to reason about shared state, coordination, contention, cancellation, and resource lifetime.

Suggested pace: 4 weeks of implementation and reasoning | Intermediate programming

What you will be able to do

- Distinguish concurrency, parallel execution, atomicity, visibility, and ordering.
- Write a bounded producer-consumer design with a clear protected predicate.
- Recognize deadlock, starvation, contention, and unsafe lifetime transitions.
- Test shutdown and failure paths instead of relying on timing sleeps.

Build the right mental model

Protect an invariant

A lock should guard a meaningful relationship, such as queue size and contents, rather than an arbitrary line of code. Reading two individually atomic fields can still observe an inconsistent combination if the invariant spans both. Define the valid state transitions first.

Waiting is conditional

A condition variable wakes a thread to recheck a predicate while holding the appropriate lock. It does not promise that the desired state remains true. Use a loop to handle spurious wakeups and competition from other threads.

Lifecycle is part of correctness

A worker may outlive the object that created it unless shutdown coordinates ownership. Stop accepting work, define drain or cancellation policy, wake waiters, join workers, and only then release resources they might still access.

Study with evidence.

Prefer deterministic barriers, explicit state transitions, and targeted stress tests to arbitrary sleeps. Keep the first implementation simple with locks; introduce lock-free structures only when a measured bottleneck and expertise justify their additional proof obligations.

Week 1: Races and invariants

Implement a shared counter and a two-field state transition. Explain why read-modify-write can race and why independent atomics may not preserve a compound invariant. Add the smallest synchronization boundary that establishes correctness.

Deliverable and check: A reproducible race demonstration and a corrected invariant with a clear happens-before argument.

Week 2: Queues and waiting

Build a bounded queue with not-empty and not-full predicates. Handle multiple producers and consumers. Validate capacity and ensure the lock is not held during the slow work itself.

Deliverable and check: Tests for empty waits, full waits, multiple workers, and preservation of every accepted item.

Week 3: Shutdown and cancellation

Add explicit closing state and decide whether queued work drains or is cancelled. Wake all affected waiters, reject new work, and join workers before destroying shared data.

Deliverable and check: A shutdown matrix covering empty, full, busy, and partially initialized states without hanging.

Week 4: Contention and design trade-offs

Measure lock hold time, throughput, and tail latency. Compare smaller critical sections, partitioned state, and batching before changing primitives. Review lock ordering and error paths.

Deliverable and check: A measured optimization that retains correctness and does not merely move the bottleneck elsewhere.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Close a bounded queue without losing accepted work

Several producers submit tasks to a capacity-limited queue and workers process them. The owner requests graceful shutdown while some producers are blocked on a full queue and some workers may be waiting for work.

A worked approach

1. Under one lock, set a closing flag and stop accepting new submissions. Document whether a submission racing with close is accepted before the linearization point or rejected afterward. Never leave its result ambiguous.
2. Notify waiting producers so they can observe closure. Workers wait while the queue is empty and not closing; when closing and empty, they exit. When work remains, they remove it under the lock and process it outside the critical section.
3. Join all workers before freeing queue storage or callbacks they use. Count accepted and completed items, inject processing failures, and define how those failures are reported without deadlocking shutdown.

Expected result

The drain policy gives every accepted item a terminal outcome, and all waiters have an exit condition. Shared memory remains alive until workers stop referencing it.

Verify the result

- Does close wake producers as well as consumers?
- Is each condition predicate rechecked in a loop?
- Can destruction occur before any worker has finished using shared state?

Extend the exercise

Add immediate cancellation as a separate policy. Report cancelled queued items explicitly, distinguish them from in-flight work, and explain what can or cannot be interrupted safely.

Make the learning stick.

Does volatile make a shared counter thread-safe?

Not in C or C++. It does not provide the required atomic read-modify-write or general synchronization guarantees. Use the language's atomic or locking primitives according to the invariant and memory model.

Why not hold the queue lock during processing?

Slow processing would serialize unrelated producers and workers, increasing contention and tail latency. Remove or reserve the item under the lock, then perform independent work after releasing it.

Can more threads reduce throughput?

Yes. Contention, cache coherence, context switching, memory pressure, and dependency limits can outweigh parallelism. Measure the actual workload and resource bottleneck before increasing concurrency.

Avoid these traps

- Using sleeps as the main correctness mechanism in concurrent tests.
- Destroying shared resources before joining workers.
- Assuming an atomic variable preserves a multi-variable invariant.

Ready for the next step?

- Each lock has a stated invariant and ownership rule.
- Condition waits have explicit predicates and closure behavior.
- Shutdown covers blocked, queued, and in-flight work.
- Performance changes are measured without weakening correctness.

Reliable next reads

- [OSTEP: concurrency chapters](#)
- [C++ Core Guidelines: concurrency](#)
- [Clang ThreadSanitizer](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.